mem9 · TiDB Cloud

TECHNICAL DEMO REPORT

Shared memory for every AI coding agent.

How a team of AI coding tools — Claude Code, Codex, and Cursor — share one queryable knowledge base on mem9, so none starts cold or repeats another's work.

Team acme **Repos** pulumi · lza **Store** mem9 → TiDB Cloud

Access Model Context Protocol

THE IDEA

Three tools, one brain.

A platform team runs its infrastructure as code — AWS, Cloudflare, and Okta via Pulumi, plus an AWS Landing Zone for accounts and guardrails. Increasingly that code is written by AI coding agents. The problem isn't any single agent's capability; it's that **they share no memory**. Each session starts cold, re-derives context from the filesystem, and re-makes the same mistakes.

Without shared memory

Duplication

An agent can't see a component already exists, so it builds a second one — a duplicate bucket, a parallel DNS record.

Broken conventions

It writes a raw `aws.s3.Bucket` instead of the library wrapper, skipping required tags and encryption.

Invisible blast radius

It changes a shared library and breaks resources two or three hops away that it never saw.

Cold starts

The next session — or the next tool — re-derives everything from scratch and repeats the same errors.

The fix. One shared, queryable memory in TiDB Cloud — reached through mem9 — that every agent reads **before** it builds and writes back **when** it's done. A single standing instruction makes the behavior automatic across all three tools: **query first → compose libraries → write back.**

TOPOLOGY

Team = space. Repo = namespace.

mem9 maps cleanly onto how teams already organize work. A **team** is a mem9 **space** — one API key, one isolated TiDB Cloud cluster behind it. A **repo** is an `appId` namespace inside that space, so multiple repos share a cluster but keep separate recall scopes.

acme · pulumi

`appId acme_pulumi_kb`

AWS · Cloudflare · Okta IaC. 21 components: reusable libraries plus production and staging instances.

acme · lza

`appId acme_lza_kb`

AWS Landing Zone: accounts, OUs, SCPs, IAM baseline. 11 components modelling the org layer.

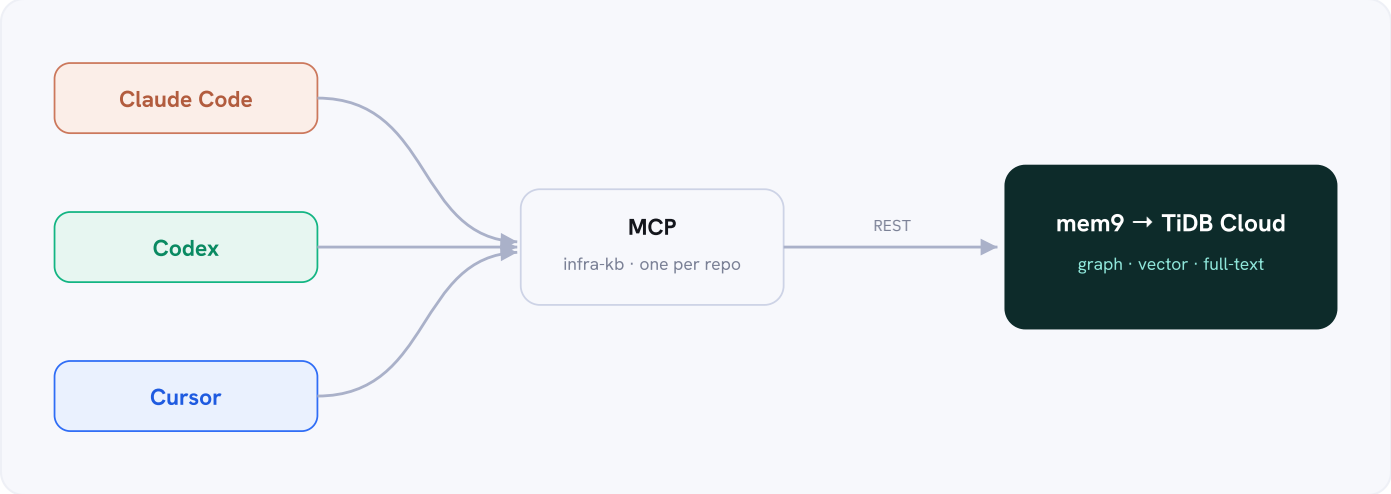
Isolation & cross-repo

- → **Cross-repo** within a team: an `account_ref` links LZA accounts to the Pulumi stacks that live in them — recalled from both namespaces and merged client-side.
- → **Cross-team** isolation: a different team is a different space with a different API key. One team cannot read another's memories — enforced by mem9 at the space level, not by application code.

In production each team's space sits on its own managed TiDB Cloud cluster. Agents never handle raw database credentials — the mem9 API key is the only secret.

ARCHITECTURE

One store. One plug. One key.



MCP is the plug

Every tool loads the same Model Context Protocol config — one named server per repo. No custom glue per tool.

One store, every pattern

Relational state, the dependency graph, vector embeddings, and full-text all live in the same memory. No ETL between four systems.

Just an API key

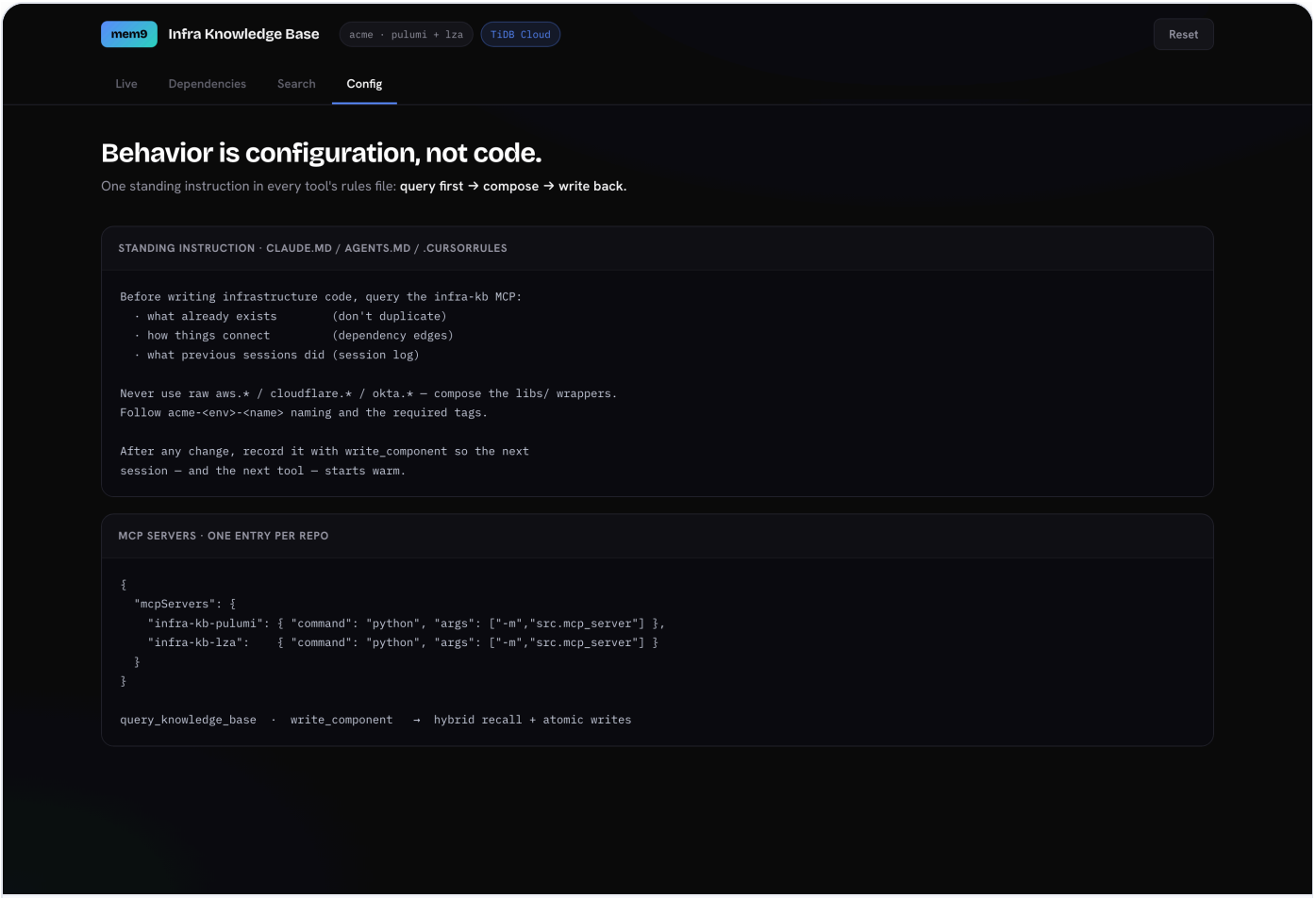
mem9 provisions and manages the TiDB Cloud cluster. Agents read and write through one key and never touch credentials.

The standing instruction

Loaded into every tool's rules file (`CLAUDE.md` / `AGENTS.md` / `.cursorrules`), so the read-first / write-back behavior happens on every run with no per-task reminder:

```
Before writing infrastructure code, query the infra-kb MCP:
  · what already exists        (don't duplicate)
  · how things connect         (dependency edges)
  · what previous sessions did (session log)

Never use raw aws.* / cloudflare.* / okta.* — compose the libs/ wrappers.
After any change, record it with write_component so the next
session — and the next tool — starts warm.
```



DASHBOARD · CONFIG The **Config** tab: the standing instruction every tool loads, and the one-entry-per-repo MCP wiring that makes it automatic.

CAPABILITY SPOTLIGHT

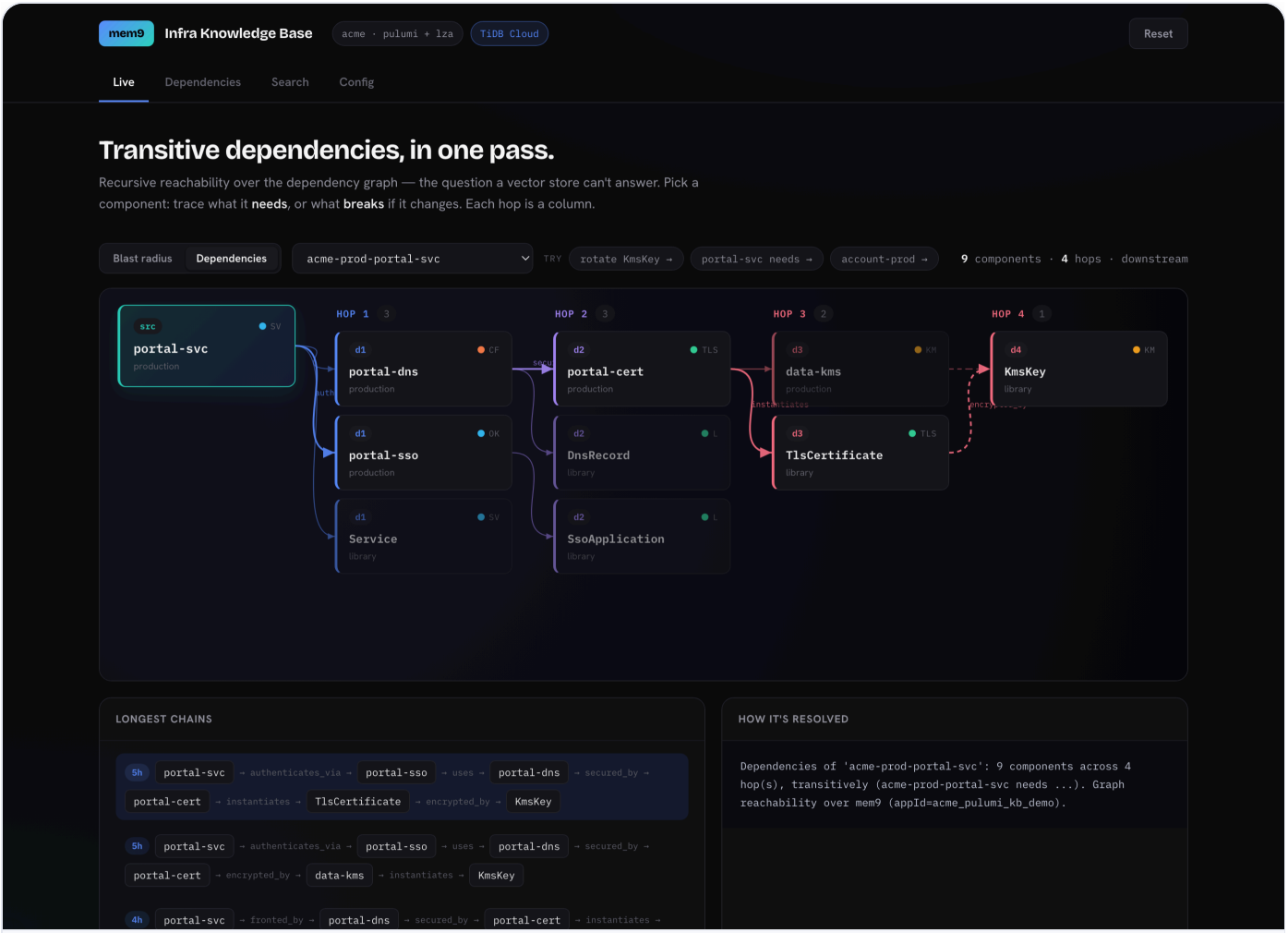
The question similarity can't answer.

A vector store finds things that *look* related. It cannot compute a transitive closure — "what, exactly, breaks if I change this?" That is graph reachability, and it is what recursive traversal over the dependency graph gives you. The knowledge base records each `depends_on` and every composition edge; mem9 holds them, and a single traversal walks the chain to any depth.

A five-hop chain

The production customer-portal service sits at the top of a chain that bottoms out at a single KMS key — five hops down, across four resource types:

`portal-svc` authenticates_via → `portal-sso` redirects_to → `portal-dns` secured_by → `portal-cert` encrypted_by → `data-kms` instantiates → **`KmsKey`**



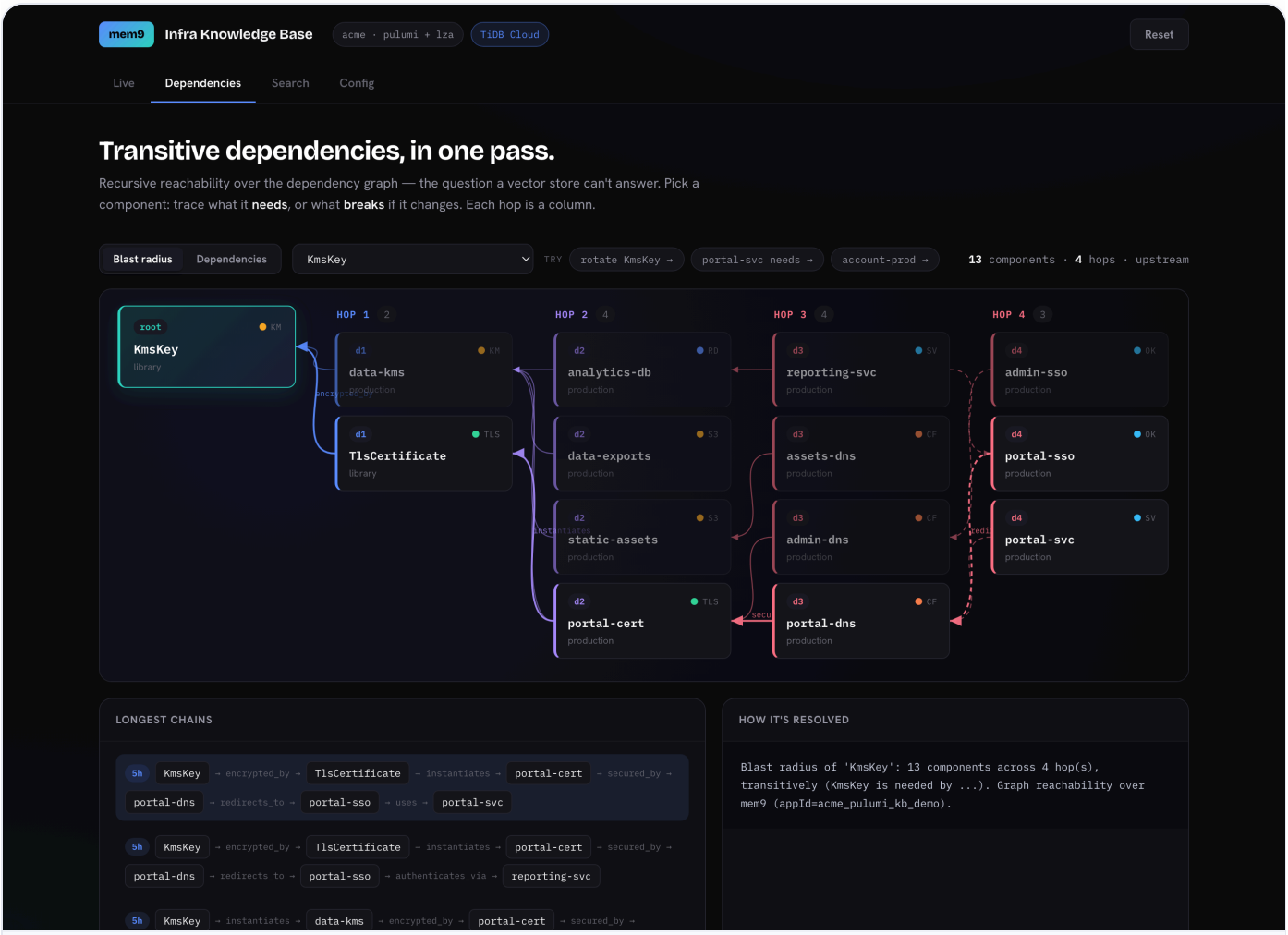
DASHBOARD · DEPENDENCIES "What does **portal-svc** need?" — each hop is a column; the chain reads left-to-right and converges on the shared KMS foundation. 9 components, 4 hops downstream.

Blast radius of one KMS key

Run the traversal in reverse from `KmsKey` — "what breaks if we rotate it?" — and the answer is **13 components across four depths**. No flat search or similarity ranking surfaces this; only walking the graph does.

DEPTH	COUNT	COMPONENTS REACHED
d1	2	data-kms, TlsCertificate
d2	4	portal-cert, analytics-db, data-exports, static-assets
d3	4	portal-dns, admin-dns, assets-dns, reporting-svc
d4	3	portal-sso, portal-svc, admin-sso

> **Why it matters.** Before touching a shared resource, an agent (or a human) sees the full transitive closure of what depends on it — and stages the change accordingly. The cost of a blind change is exactly the surprise this prevents.



DASHBOARD · DEPENDENCIES "What breaks if we rotate **KmsKey**?" — the reverse traversal fans 13 components across four depths, all funneling onto the anchored foundation. The Trace panel reads each chain as a sentence.

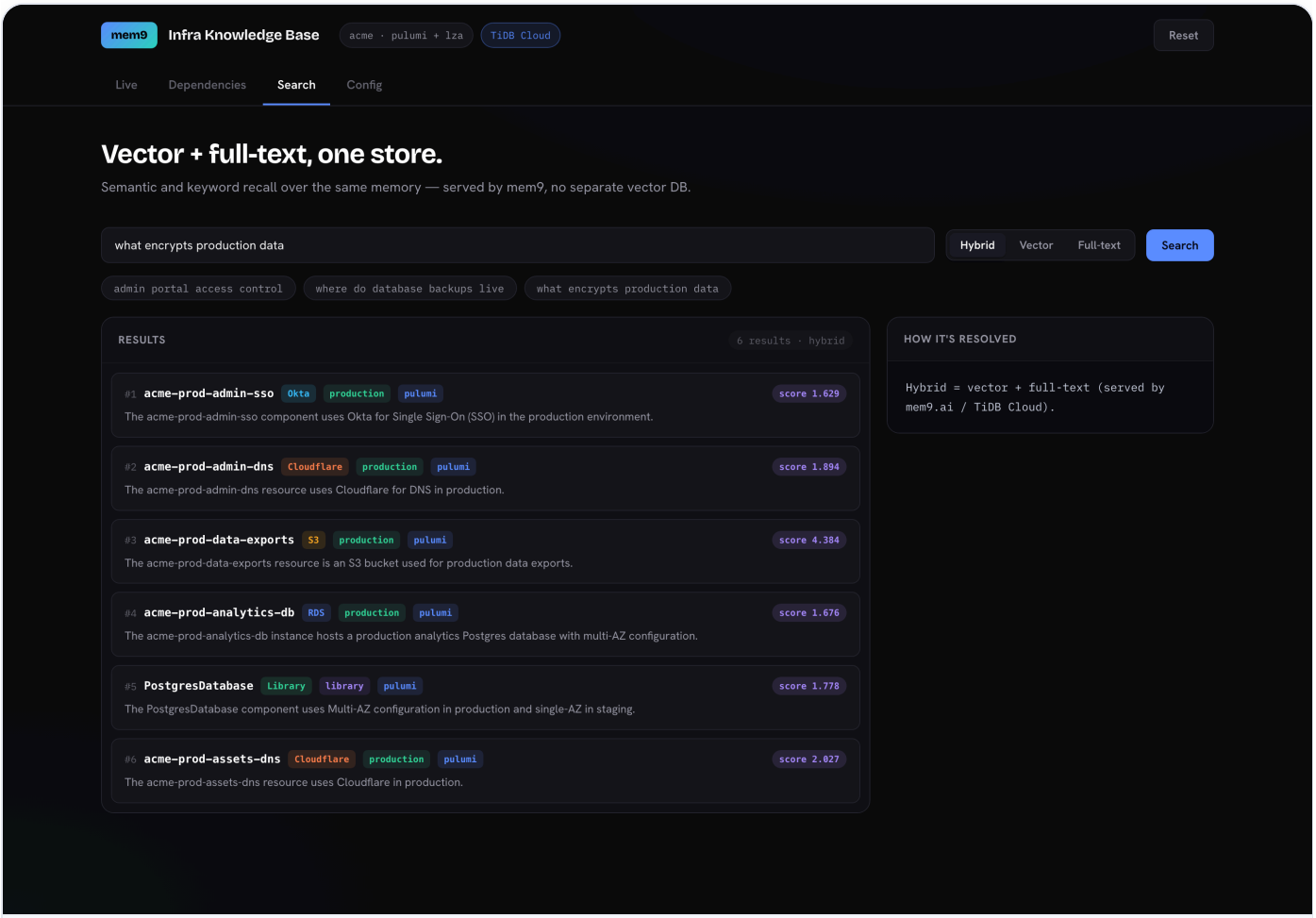
The live dashboard renders this as a depth-laned "Dependency Rail": each hop is a column, connectors carry the relationship, and the foundation node is anchored as everything funnels toward it.

RECALL & RATIONALE

Hybrid recall, and why one engine.

Vector + full-text, one store

The same memory that holds the graph also answers semantic and keyword queries. Ask *"what encrypts production data"* and mem9 returns the KMS key, the resources it encrypts, and the TLS cert — ranked by hybrid relevance, served from TiDB Cloud with no separate vector database and no synchronization to keep in step.



DASHBOARD · SEARCH The **Search** tab: hybrid (vector + full-text) recall over the same store, with per-result type, environment, and repo, scored by relevance.

Why mem9 on TiDB — versus the alternatives

APPROACH	GRAPH REACHABILITY	SEMANTIC RECALL	ATOMIC AGENT STATE
S3 / filesystem	no traversal	no	no transactions
Vector DB only	similarity ≠ reachability	yes	no
Four separate systems	partial, with ETL	yes	no cross-system atomicity
mem9 → TiDB Cloud	recursive traversal	vector + full-text	one transaction

Component, edge, and session-log entry are written together. Relational state, the dependency graph, vector embeddings, and full-text indexes are one engine — so the agent's memory is consistent across concurrent writes from all three tools.

> **S3 is the filing cabinet for artifacts. mem9 is the brain agents query.**

THE LIVE DEMO

Closing the staging gap, warm.

Production is complete; staging has drifted — it's missing its DNS and SSO layer (four components). Three different tools bring staging to parity, each reading the shared memory before it builds and writing back when it's done. No agent starts cold.

1 · Claude Code

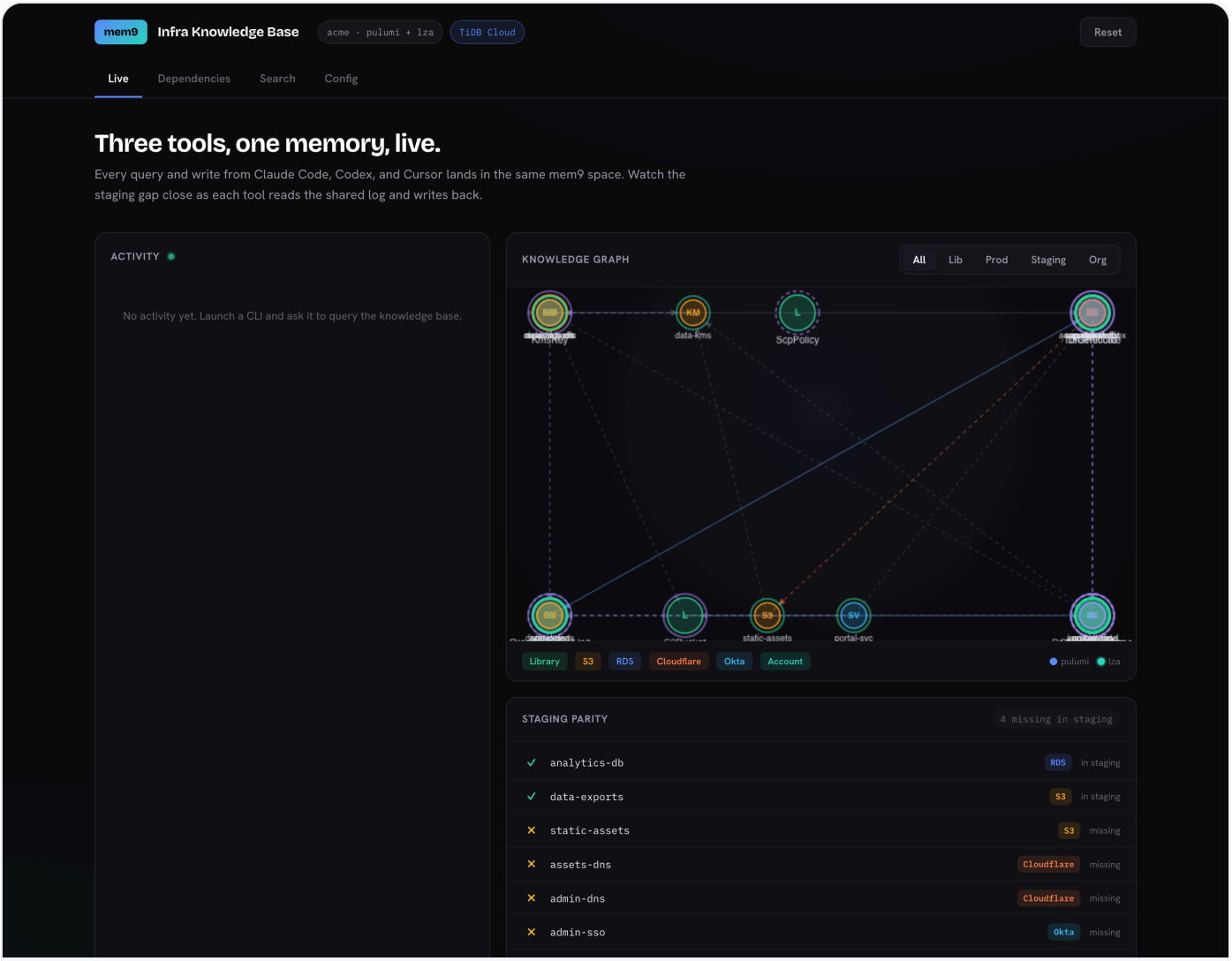
Queries the KB, finds staging missing static-assets + DNS, composes the libraries, writes both back. **Gap 4 → 2.**

2 · Codex

Opens cold with zero context. Reads the shared session log, sees what was just built, continues with admin-DNS. **Gap 2 → 1.**

3 · Cursor

Traces what prod's admin-SSO depends on, confirms it must redirect to a DnsRecord, builds the matching SSO app. **Gap 1 → 0.**



DASHBOARD · LIVE The **Live** tab: the activity feed (every tool's reads and writes), the shared knowledge graph, and the staging-parity strip — four components missing, before the agents run.

Each tool attributes its work in the activity feed; the parity strip turns green as the gap closes. The same component graph, dependency traversal, and hybrid search shown in this report are all live views over the one mem9 space — the dashboard is an observability window on the system, not the system itself.

---

mem9 · TiDB Cloud — shared memory for AI coding agents. Generated for the team/repo demo (team acme, repos pulumi + lza).